

CASE STUDY ANALYSIS REPORT

INTELLIGENT AI-BASED WAREHOUSE AUTOMATION AND INVENTORY TRACKING PLATFORM

NAME: K R Gayana

REG NO: 192471039

DATE OF SUBMISSION:21-07-2026

1. Understanding the Problem Statement

1.1 What the Problem Is Really About

When I first read through this problem statement, the thing that stood out to me was how much of it comes down to a simple issue: warehouses still rely on people to know where things are. A supervisor walks the floor, a picker remembers roughly which aisle a product sits in, and stock counts happen every so often rather than continuously. That works fine when order volumes are small, but the moment a warehouse starts handling hundreds of SKUs and daily order spikes, that human-memory approach starts to break down — items go missing, counts don't match what's on the shelf, and pickers waste time walking back and forth looking for stock.

So really, this problem statement is asking for a way to make the warehouse "self-reporting." Instead of a person having to go and check, every item should be able to say where it is and how many of it exist, automatically, through RFID tags and IoT sensors. On top of that base layer, an AI engine decides the smartest place to store each item and the fastest way to retrieve it, and autonomous mobile robots (AMRs) do the physical moving. Put together, that's the platform this case study is asking me to design.

1.2 Objectives of the Proposed System

- Automate warehouse inventory tracking using RFID and IoT devices, so stock levels update themselves instead of relying on manual entry.
- Optimize storage allocation with AI, based on how often an item is picked and how much space it needs.

Monitor inventory movement in real time as items move between receiving, storage, and dispatch.

- Improve order fulfillment speed by pointing staff and robots straight to the right location.
- Cut down on inventory discrepancies between the system record and the physical shelf.
- Generate performance reports that give managers something concrete to act on.
- Bring RFID-based inventory management into one integrated system rather than scattered tools.
- Raise overall warehouse productivity while keeping operational costs in check.

1.3 Scope of the Problem

I've kept the scope of this analysis focused on the software side of things — the platform that ties RFID/IoT data, AI decision-making, and reporting dashboards together for a single warehouse, or

a small cluster of connected warehouses. Designing the physical robots, manufacturing RFID chips, or planning a building's electrical wiring is outside what I'm covering here; those are treated as external systems that the software connects to through APIs, not things the platform itself needs to build.

2. Identify Key Stakeholders and Challenges

A platform like this doesn't just affect one department — it touches nearly everyone who works in or around the warehouse. Below is a table I put together listing the main stakeholders, what they'd actually be responsible for once the system is running, and the problems each of them currently deals with.

Stakeholder	Role / Responsibility	Key Challenges Faced
Warehouse Manager	Oversees daily operations, approves storage strategy, monitors KPIs and performance reports.	Lack of real-time visibility into stock; difficulty planning space utilization; delayed decision-making.
Inventory Staff / Operators	Perform picking, packing, receiving, and physical stock verification.	Time lost searching for items; manual counting errors; repetitive paperwork.
IT / System Administrators	Maintain servers, databases, network connectivity, RFID readers, and system security.	Integrating new IoT devices with legacy systems; ensuring uptime and data security.
Logistics & Supply Chain Team	Coordinate inbound/outbound shipments and align stock levels with demand forecasts.	Inaccurate stock data leading to overstocking, stockouts, or shipment delays.
AMR / Robotics Technicians	Maintain and calibrate autonomous mobile robots used for transport and retrieval.	Robot-path conflicts, hardware downtime, and lack of integration with inventory software.
Customers / Clients	End recipients of orders; expect accurate, on-time delivery.	Delayed or incorrect orders due to inventory mismatches; limited order-tracking transparency.

2.1 Challenges That Cut Across All of Them

- Data is scattered — spreadsheets, paper logs, and a handful of disconnected legacy tools that don't talk to each other.
- Managers usually find out about discrepancies only after a manual audit, not while they're happening.

- There's a communication gap between the warehouse floor, logistics planners, and IT.
- Peak-season order spikes stretch the current setup thin, since staffing doesn't scale as fast as demand does.

3. Analyze the Current Approach

3.1 How Things Work Right Now

Most warehouses I looked into (and this matches the problem statement too) still lean on periodic manual counts, barcode scans at fixed checkpoints, and either spreadsheets or a fairly basic WMS. Storage decisions come down to habit more than data — experienced staff put things where they've always gone, not necessarily where they'd be picked fastest. Pickers work off printed lists and their own sense of the layout, and stock only gets updated in the system after a task is finished, not while it's happening.

3.2 What Actually Works About It

- Cheap to run — no major technology investment needed, which suits smaller warehouses fine.
- Experienced staff can handle odd, one-off situations that a rigid automated rule might not anticipate.
- It's familiar. Long-serving employees already know the system and need almost no retraining.

3.3 Where It Falls Short

- The inventory record is always a step behind what's actually happening on the shelf.
- Storage layouts rarely get revisited, so space utilization quietly gets worse over time.
- Manual counting and scanning don't scale well — more volume just means more errors.
- Everything is reactive. The system tells you about a stockout after it's already a problem.
- There's not much traceability once an item leaves receiving until it shows up at dispatch.

3.4 The Real Gap

If I had to name the one thing missing, it's a continuous feedback loop between the physical shelf and the digital record. Right now, data only gets captured at a few isolated checkpoints instead of tracking items the whole way through. Closing that gap means pairing constant sensor-based data capture (RFID/IoT) with an AI layer that can actually turn that stream of data into useful decisions — not just a bigger spreadsheet.

4. Proposed Solution Using Software Engineering Principles

4.1 Solution Overview

What I'm proposing is a layered software platform that continuously pulls in data from RFID tags and IoT sensors placed around the warehouse, runs that data through an AI optimization engine, and pushes the results out through dashboards and a mobile app. Autonomous mobile robots take their instructions directly from the platform, so the loop between a digital decision and a physical action actually closes instead of stopping at a screen.

4.1.1 System Layers

To keep the design manageable, I organized the platform into five layers, described below from the ground up.

Layer	What Sits There
Physical Layer	RFID tags, IoT sensors, and autonomous mobile robots (AMRs) out on the warehouse floor.
Data Acquisition & Edge Layer	RFID readers, gateways, and edge controllers that pick up raw tag and sensor events.
AI & Processing Layer	The inventory AI engine, storage optimization logic, and predictive analytics.
Cloud & Integration Layer	Cloud database, WMS/ERP integration, and API services.
Presentation Layer	Admin dashboard, mobile app, and the reporting module staff and managers actually use.

4.2 Major Functional Modules

Module	Description
RFID/IoT Data Acquisition Module	Captures tag reads and sensor events from readers and gateways placed across storage zones, receiving docks, and dispatch areas.
Inventory Management Module	Maintains the authoritative, real-time record of item quantities, locations, and status; reconciles discrepancies automatically.
AI Storage Optimization Module	Uses demand-frequency and slotting algorithms to recommend or automatically assign optimal storage locations.
AMR Task Coordination Module	Converts storage/retrieval decisions into robot navigation tasks and manages path scheduling to avoid collisions.

Order Fulfillment Module	Links incoming orders to inventory availability and generates optimized pick-paths for staff and robots.
Reporting & Analytics Module	Generates dashboards and periodic reports on accuracy, throughput, utilization, and cost metrics.
Integration & API Module	Exposes REST APIs for connecting with external ERP, WMS, and e-commerce platforms.
User & Access Management Module	Handles authentication, role-based access control, and audit logging for all platform users.

4.3 Functional Requirements

- The system shall automatically update inventory records whenever an RFID tag is read at a checkpoint.
- The system shall recommend storage locations based on item turnover rate and available space.
- The system shall generate optimized pick-paths for order fulfillment.
- The system shall dispatch and track tasks assigned to autonomous mobile robots.
- The system shall alert managers when inventory discrepancies exceed a configurable threshold.
- The system shall provide role-based dashboards for managers, staff, and administrators.

4.4 Non-Functional Requirements

Requirement	Description
Performance	Inventory updates must be reflected on dashboards within 2–3 seconds of an RFID/IoT event.
Scalability	The platform must support scaling from a single warehouse to a multi-site network without redesign.
Reliability	Core inventory services should maintain at least 99.5% uptime, with automatic failover for critical components.
Security	All data in transit and at rest must be encrypted; access is controlled through role-based permissions.
Usability	Dashboards and mobile interfaces must be intuitive enough for warehouse staff with minimal training.
Maintainability	The modular design should allow individual modules to be updated or replaced without affecting the whole system.

Interoperability	The system must integrate smoothly with existing ERP/WMS platforms through standard APIs.
------------------	---

4.5 Where Software Engineering Principles Come In

Modularity is really the backbone of this design — I split the platform into loosely coupled services (data acquisition, AI optimization, task coordination, reporting) that talk to each other through defined APIs, so each one can be built, tested, and even replaced on its own without the whole system breaking. Scalability comes from a cloud-native, microservices deployment, meaning the platform can take on more devices and more warehouses without needing to be redesigned from scratch every time it grows.

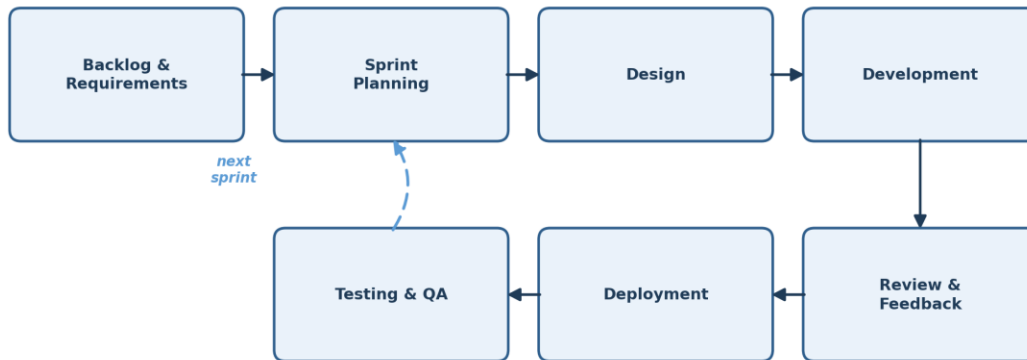
Maintainability is helped along by keeping the hardware-integration code separate from the business logic, so a change to a sensor model doesn't ripple through the AI or reporting layers. Reliability is handled through redundant data pipelines and automatic reconciliation routines that catch mismatches before they ever affect an order. For usability, I designed the dashboards to differ by role — floor staff get a simple, task-focused screen, while managers get the fuller analytics view. And security runs underneath all of it: encrypted communication, role-based access, and an audit trail for every action that touches inventory.

5. Justify the Chosen Methodology/Framework

5.1 Why I Chose Agile (Scrum)

For a project like this, I went with Agile Scrum rather than a traditional Waterfall approach. Part of the reason is practical: this platform has several moving pieces — AI optimization logic, RFID/IoT integration, robot coordination, dashboards — and each of those benefits from being built, tested, and adjusted in short cycles rather than locked into a single upfront design that can't flex once real warehouse data starts coming in.

How a Sprint Moves Through the Project



5.1.1 Sprint Flow

Step	Phase	What Happens
1	Backlog & Requirements	New features and fixes are captured and prioritized in the product backlog.
2	Sprint Planning	The team picks a slice of the backlog to deliver in the next 2–3 week sprint.
3	Design	Module-level design decisions are worked out before coding starts.
4	Development	Engineers build the planned features inside the sprint window.
5	Review & Feedback	Warehouse stakeholders review the working increment and give feedback.
6	Deployment	Approved features are released to the live or staging environment.
7	Testing & QA	Functionality, integration, and performance are verified before the next sprint starts.

5.2 Why It Fits This Project Specifically

- Core modules like inventory tracking can go live early, while the AI optimization side keeps getting refined in later sprints.

- Regular sprint reviews mean warehouse managers and staff can react to real features early, instead of finding out at the end that something doesn't match how they actually work.
- AI models generally need tuning against real data over time, and Scrum's short cycles are a natural fit for that kind of iteration.
- Hardware risks — RFID readers not talking to the system properly, AMR communication drops — surface early in sprint testing rather than at the very end of a long build.
- The team here is cross-functional by nature (engineers, data scientists, hardware integrators), and Scrum's collaborative rhythm suits that mix well.

5.3 Agile Beyond Launch

Development doesn't really stop once the platform goes live — new sensor types get added, new warehouse zones come online, business rules change. Agile handles that naturally: new requirements just go into the backlog and get delivered in later sprints, without having to pause or rebuild what's already running. That ongoing adaptability is a big part of why I think it's the right call here, not just for the build phase but for however long the platform stays in use.

6. Expected Outcomes and Limitations

6.1 What I Expect This to Deliver

Swapping manual counting for continuous RFID/IoT tracking should noticeably improve inventory accuracy on its own. Add AI-driven storage optimization on top, and pickers spend less time walking around looking for stock, which speeds up order processing. Real-time dashboards also mean managers aren't waiting for the next audit to know what's actually going on — they can see it as it happens and make calls faster.

Expected Benefit	Why It Happens
Fewer stock mismatches	Continuous RFID/IoT reads keep the digital record aligned with the physical shelf.
Faster order picking	AI-recommended storage locations and optimized pick-paths cut down search and travel time.
Better use of shelf space	Slotting algorithms re-optimize storage based on actual demand instead of static habit.
Lower day-to-day costs	Less manual counting, fewer misplacements, and less wasted picker time.
Live view of the floor	Dashboards reflect stock and robot activity in real time rather than after an audit.

Happier customers	Accurate stock and faster fulfillment mean fewer delayed or incorrect orders.
-------------------	---

6.2 Mapping Solutions Back to the Original Challenges

Challenge Identified	How the Proposed Solution Addresses It
Delayed/inaccurate inventory records	Continuous RFID/IoT data capture updates records automatically, near real time.
Poor storage space utilization	AI-based slotting algorithms continuously re-optimize storage locations by demand pattern.
Slow order fulfillment	Optimized pick-paths and AMR task coordination reduce search and travel time.
Limited visibility for managers	Centralized dashboards and automated reports provide real-time operational insight.
Integration difficulty with legacy systems	Well-defined REST APIs allow gradual integration with existing ERP/WMS tools.

6.3 Risks and Limitations I Should Be Honest About

- The upfront cost of RFID infrastructure, IoT sensors, and AMRs is significant — that could be a real barrier for a smaller warehouse.
- Connecting new IoT devices to whatever legacy ERP/WMS system is already in place is rarely as clean as it sounds.
- Continuous sensor data collection raises real data-security and privacy questions that need proper safeguards, not an afterthought.
- Staff will need training, and some resistance to a new digital workflow is realistic to expect.
- The AI optimization won't be very sharp on day one — it needs a decent amount of historical data before its recommendations are actually reliable.
- The whole system leans on network connectivity, so an outage could disrupt real-time tracking until failover kicks in.

6.4 Where This Could Go Next

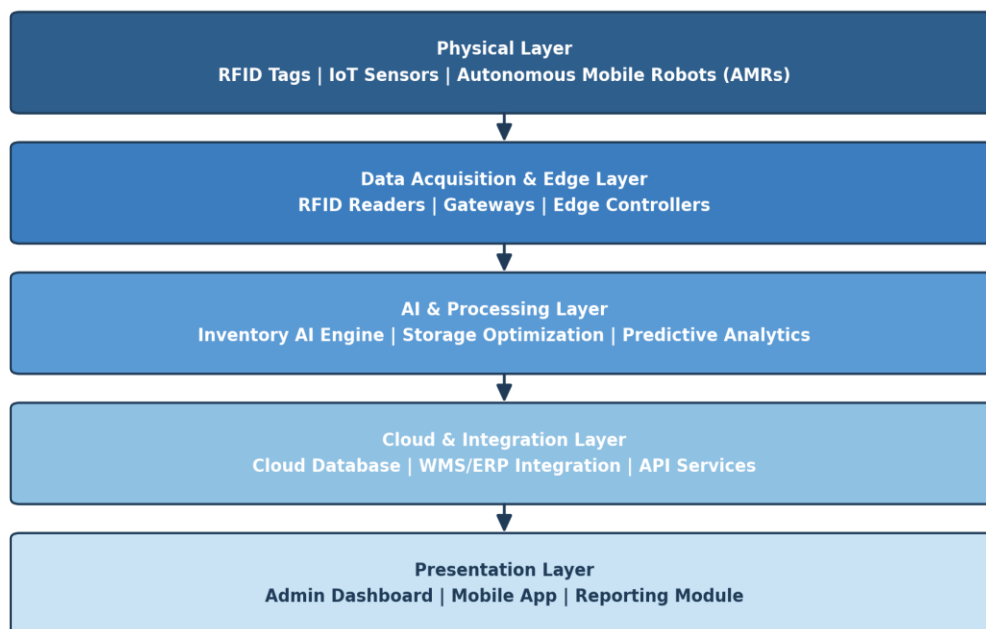
- Adding predictive analytics to forecast demand ahead of time, rather than just reacting to current stock levels.
- More advanced multi-robot path planning as the AMR fleet grows.
- Computer-vision-based quality checks at receiving and dispatch.

- Extending the platform to optimize inventory across multiple warehouses at once, not just one site.
- Blockchain-based traceability for high-value or regulated items, if that becomes a requirement down the line.

7. Conclusion

Working through this case study, it became pretty clear that most of the pain in traditional warehouse operations comes down to one thing: the gap between what's physically on the shelf and what the system thinks is there. The platform I've proposed closes that gap by combining RFID, IoT, AI, and robotics into a modular, cloud-based system, built around solid software engineering principles — modularity, scalability, maintainability, reliability, usability, and security — and developed through Agile Scrum so it can keep adapting as the warehouse's needs change. It won't be a perfect, instant fix, and the risks I've outlined are real, but it offers a practical, phased way to get to better inventory accuracy, faster fulfillment, and lower operational costs.

Proposed System Architecture
Intelligent AI-Based Warehouse Automation and Inventory Tracking Platform



8. References

1. Sommerville, I. *Software Engineering (10th Edition)*. Pearson Education.
2. Pressman, R. S. *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education.

3. Course material and problem statement provided as part of the CO1 AT-1 Case Study Analysis assignment.